7.1. Implémentation du service générique

1. Sous le dossier **Interfaces** du projet **AM.ApplicationCore**, créer l'interface générique « **IServices** ».

```
namespace AM.ApplicationCore.Interfaces
{
    public interface IService<T> where T : class
    {
        void Add(T entity);
        void Update(T entity);
        void Delete(T entity);
        void Delete(Expression<Func<T, bool>>
where);
        T GetById(object id);
        T GetById(string id);
        IEnumerable<T> GetMany(Expression<Func<T,
bool>> where = null);
        T Get(Expression<Func<T, bool>> where);
        void Commit();
    }
}
```

2. Sous le dossier **Services** du projet **AM.ApplicationCore**, créer la classe **Service** générique qui implémente l'interface **IService**.

```
namespace AM.ApplicationCore.Services
{
    public class Service<T> : IService<T>
where T : class
    {
        private readonly IGenericRepository<T>
_repository;
        private readonly IUnitOfWork
_unitOfWork;
        public Service(IUnitOfWork unitOfWork)
        {
            this._repository =
unitOfWork.Repository<T>();
            this._unitOfWork = unitOfWork;
        }
        public virtual void Add(T entity)
        {
            _repository.Add(entity);
        }
        public virtual void Update(T entity)
        {
            _repository.Update(entity);
        }
        public virtual void Delete(T entity)
        {
            _repository.Delete(entity);
        }
        public virtual void
Delete(Expression<Func<T, bool>> where)
        {
            _repository.Delete(where);
        }
        public virtual T GetById(object id)
        {
            return _repository.GetById(id);
        }
        public virtual T GetById(string id)
        {
            return _repository.GetById(id);
        }
        public virtual IEnumerable<T>
GetMany(Expression<Func<T, bool>> filter =
null)
        {
            return
_repository.GetMany(filter);
        }
        public virtual T
Get(Expression<Func<T, bool>> where)
        {
            return _repository.Get(where);
        }
        public void Commit()
        {
            try
            {
                _unitOfWork.Save();
            }
            catch (Exception ex)
            {
                throw;
            }
        }
    }
}
```

7.2. Implémentation du service spécifique

3. Modifier toutes les interfaces de Services Spécifiques, I**ServicePlane** et I**ServiceFlight,** afin qui ils héritent de l'interface du service générique « I**Service** ».

```
namespace AM.ApplicationCore.Interfaces
{
    public interface IServicePlane : IService<Plane>
    {

    }
}
namespace AM.ApplicationCore.Interfaces
{
    public interface IServiceFlight : IService<Flight>
    {

    }
}
```

4. Modifier toutes les classes de Services Spécifiques, **ServicePlane** et **ServiceFlight,** afin qui ils héritent de la classe du service générique « **Service** ».

```
namespace AM.ApplicationCore.Services
{
    public class ServicePlane : Service<Plane>, IServicePlane
    {
        private readonly IUnitOfWork _unitOfWork ;
        public ServicePlane(IUnitOfWork unitOfWork) : base(unitOfWork)
        {
            _unitOfWork  = unitOfWork ;
        }
    }
}

namespace AM.ApplicationCore.Services
{
    public class ServiceFlight : Service<Flight>, IServiceFlight
    {
        private readonly IGenericUnitOfWork <Flight> _unitOfWork ;
        public ServiceFlight(IGenericUnitOfWork <Flight> unitOfWork) : base(unitOfWork)
        {
            _unitOfWork  = unitOfWork ;
        }
    }
}
```

5. Modifier les méthodes de la classe **ServiceFlight** pour qu'elles utilisent des données de la base de données.

```
namespace AM.ApplicationCore.Services
{
    public class ServiceFlight : Service<Flight>, IServiceFlight
    {
        public List<Flight> Flights => GetMany().ToList();

        ...

    }
}
```

6. Dans la classe **ServicePlane**, implémenter les services suivants :

   - Le service **GetPassenger** qui retourne les voyageurs d'un avion passé en paramètre.

   - Le service **GetFlights** qui retourne les vols ordonnés date de départ des n derniers avions.

   - Le service **IsAvailablePlane** qui retourne true si on peut réserver n place à un vol passé en paramètre.

   - Le service **DeletePlanes** qui supprime tous les avions dont la date de fabrication a dépassé 10 ans.

7. Tester les méthodes implémentées dans la classe **ServicePlane**.

```
namespace AM.ApplicationCore.Interfaces
{
    public interface IServicePlane : IService<Plane>
    {
        IList<Passenger> GetPassengers(Plane plane);
        IList<IGrouping<int, Flight>> GetFlights(int n);
        bool AvailablePlane(Flight flight, int n);
        void DeletePlanes();
    }
}
```

```
namespace AM.ApplicationCore.Services
{
    public class ServicePlane : Service<Plane>, IServicePlane
    {
        public ServicePlane(IUnitOfWork unitOfWork) : base(unitOfWork) {        }

        public bool AvailablePlane(Flight flight, int n)
        {
            int capacity = Get(p => p.Flights.Contains(flight) == true).Capacity;
            int nbPassengers = flight.Tickets.Count();
            return capacity >= (nbPassengers + n);
        }

        public void DeletePlanes()
        {
            foreach (var plane in GetMany().Where(p => (DateTime.Now - p.ManufactureDate).TotalDays >
365 * 10).ToList())
            {
                Delete(plane);
                Commit();
            }
        }

        public IList<IGrouping<int, Flight>> GetFlights(int n)
        {
            return GetMany().OrderByDescending(p => p.PlaneId).Take(n).SelectMany(p =>
p.Flights).GroupBy(f => f.Plane.PlaneId).ToList();
        }

        public IList<Passenger> GetPassengers(Plane plane)
        {
            return GetById(plane.PlaneId).Flights.SelectMany(f => f.Tickets.Select(t =>
t.Passenger)).ToList();
        }
    }
}
```